What is a multidimensional array?

Definition (simple): A multidimensional array is an array that has **more than one axis (dimension)**. Instead of a straight line of values, you get **tables, grids, or stacks**.

- 1D → line
- 2D → table (rows × columns)
- 3D → stack of tables

NumPy stores all of this **in memory efficiently** and operates on it at once.

Real-life mental model

Think in terms of **data organization**, not math.

- **1D** → Daily temperatures for 7 days
- **2D** → Temperatures of 7 days × 3 cities
- **3D** → 7 days × 3 cities × 24 hours

Each added dimension answers **one more question**.

Example: 2D (most common)

Scenario: Marks of 3 students in 4 subjects.

```
import numpy as np

marks = np.array([
    [85, 90, 78, 88],   # Student 1
    [72, 80, 75, 70],   # Student 2
    [90, 92, 95, 93]    # Student 3
])
```

Shape:

```
marks.shape
```

Output:

```
(3, 4)
```

Meaning:

- 3 rows (students)
- 4 columns (subjects)

Example: 3D (stacked data)

Scenario: Temperature data for **2 cities**, **3 days**, **4 readings per day**.

```
temps = np.array([
    [   # City 1
        [30, 32, 31, 29],
        [31, 33, 32, 30],
        [29, 31, 30, 28]
    ],
    [   # City 2
        [25, 26, 27, 24],
        [26, 28, 27, 25],
        [24, 25, 26, 23]
    ]
])
```

Shape:

```
temps.shape
```

Output:

```
(2, 3, 4)
```

Meaning:

- 2 cities
- 3 days
- 4 readings per day

Key rule (critical)

Each dimension answers **one independent question**.

Bad thinking:

> "More dimensions = more complex"

Correct thinking:

> "More dimensions = better structured information"

Why NumPy uses multidimensional arrays

- Fast batch operations
- No nested Python loops
- Perfect for ML, images, simulations, matrices

Example (no loops):

```
temps.mean(axis=2)  # average temperature per day
```

One-line takeaway

A multidimensional array is **structured data packed into one object**, so the computer can process it **all at once, fast**.

Once this clicks, NumPy stops feeling abstract and starts feeling obvious.